

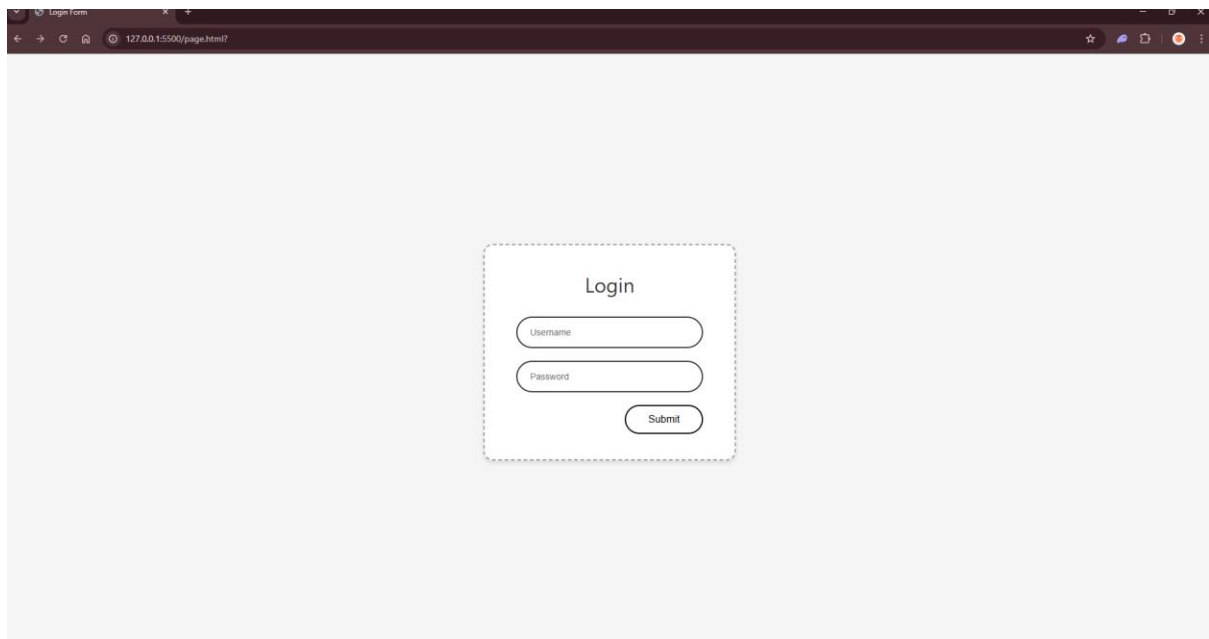
TP_MOD_04_103032300013_SyahrilArfianAlmazril
Pemrograman Web

Nama : Syahril Arfian Almazril
Nim : 103032300013
Kelas : IT-47-04

SourceCode : <https://github.com/Arfazrll/TP-WebDev>

Tahap 1 [Membuat halaman login]:

Buatlah sebuah form login sesuai dengan gambar berikut Posisikan form ditengah halaman seperti gambar berikut!



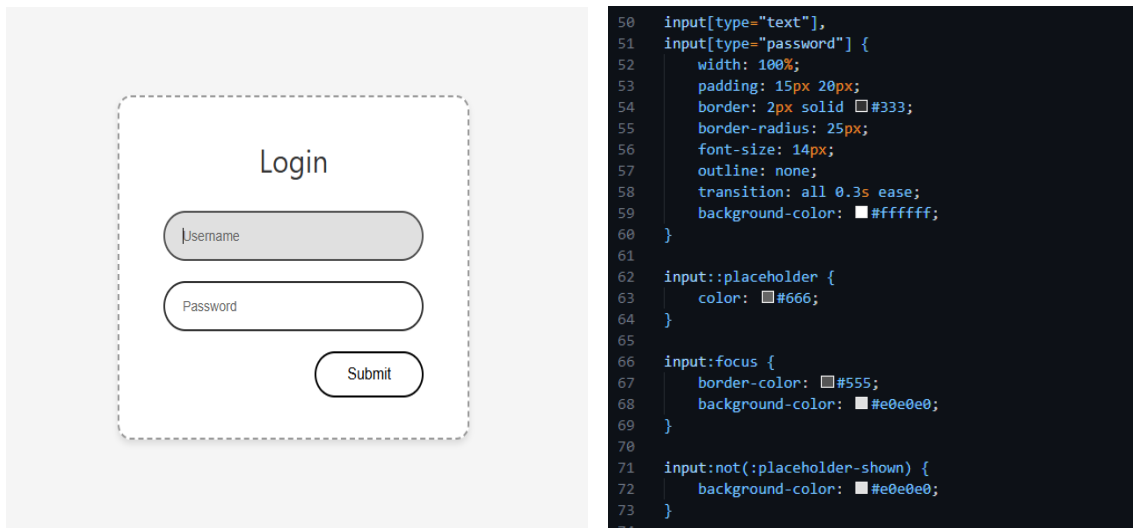
Jelaskan cara yang anda gunakan untuk menengahkan form login tersebut!

- Form login ditengahkan menggunakan **CSS Flexbox** pada elemen `<body>`. Properti `display: flex;` kemudian `justify-content: center;` menempatkannya di tengah secara horizontal, dan `align-items: center;` menempatkannya di tengah secara vertikal. Lalu Kunci agar perataan vertikal berfungsi adalah `min-height: 100vh;`, yang memastikan `<body>` memenuhi seluruh tinggi layar. Ini akan membuatnya berada selalu ditengah

```
body {  
  font-family: Arial, sans-serif;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 100vh;  
  background-color: #f5f5f5;  
}  
  
.container {  
  width: 100%;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Setelah itu, ganti warna background dari element input ketika kita mengklik input tersebut seperti berikut! (warna terserah sesuai dengan kekreativitasan masing-masing individu)
Jelaskan apa yang anda gunakan untuk membuat hal tersebut!

- Efek ini dibuat menggunakan *class* `input:focus` di CSS saya. Saat sebuah kolom input diklik, `input:focus` langsung aktif dan properti `background-color` mengubah warna latar belakangnya menjadi abu-abu.



Setelah itu, ketika tombol submit di-hover, ubah warna background menjadi hitam dan warna text menjadi putih dan ketika tombol ditekan, akan berpindah ke halaman lain. (Bebaslah yang penting berubah pas di-hover)

- Efek visual tombol diatur oleh CSS, di mana selector `:hover` digunakan untuk mengubah `background-color` menjadi hitam dan color teks menjadi putih saat kursor berada di atasnya. sedangkan pindah halaman saat tombol ditekan diatur oleh HTML. Tombol dengan `type="submit"` secara otomatis akan memicu atribut `action="section.html"` pada `<form>`, yang memerintahkan browser untuk membuka halaman tersebut.



Tahap 2.A [Membuat Navbar]:

Buatlah sebuah navigation bar yang berada selalu diatas halaman

(Tampilan dibebaskan untuk setiap individu yang terpenting mengandung sebuah logo disisi kiri dan minimal 3 navigation items pada sisi kanan)



Jelaskan apa yang anda gunakan untuk membuat logo dan items navigation dapat menjauh satu sama lain.

- Untuk memisahkan logo dan menu navigasi, saya menggunakan CSS Flexbox pada kontainer `.nav-wrapper`. Dengan menerapkan `display: flex;` dan `justify-content: space-between;`, logo di sisi kiri dan menu di sisi kanan secara otomatis akan saling menjauh. Properti `align-items: center;` juga digunakan agar keduanya sejajar secara vertikal.

```
22 .navbar {
23   background-color: transparent;
24   padding: 30px 0;
25   position: fixed;
26   width: 100%;
27   top: 0;
28   z-index: 1000;
29 }
30
31 .nav-wrapper {
32   max-width: calc(100% - 120px);
33   margin: 0 60px;
34   padding: 18px 50px;
35   display: flex;
36   justify-content: space-between;
37   align-items: center;
38   background-color: #ffffff;
39   border-radius: 60px;
40   box-shadow: 0 5px 25px rgba(0,0,0,0.1);
41 }
```

Tambahkan efek agar navigation item dapat berubah warna ketika kita menghover.

- Efek hover pada menu navigasi dibuat dengan CSS menggunakan *pseudo-class* `:hover`. Awalnya, teks menu diatur berwarna hitam. Saat *mouse* diarahkan ke salah satu item menu, *pseudo-class* `.nav-menu li a:hover` mengubah warnanya menjadi merah. Agar perubahan warna ini tidak terjadi secara instan, properti `transition: color 0.3s ease;` ditambahkan pada *style* awal, sehingga transisi warnanya terlihat halus selama 0.3 detik.

```
.nav-menu li a {
  color: #000000;
  text-decoration: none;
  font-size: 19px;
  font-weight: 500;
  transition: color 0.3s ease;
  padding: 10px 0;
  display: inline-block;
}

.nav-menu li a:hover {
  color: #e74c3c;
}
```

Tahap 2.B [Membuat Section Home]:

Buatlah section home seperti berikut! (Tampilan masih dibebaskan untuk setiap individu yang terpenting memiliki sebuah box dengan background sebuah gambar [sangat dilarang menggunakan gambar yang mengandung sara dan lain sebagainya] dan kalimat yang memiliki sebuah kata yang memiliki font size dan warna yang berbeda dan posisinya harus ditengah sesuai dengan gambar)



Tahap 2.C [Membuat Section Collections]:

Buatlah section collection sebagai berikut! (Dibebaskan dengan catatan menggunakan gambar yang tidak mengandung sara dan lain sebagainya dan minimal mencantumkan 4 gambar berbeda secara horizontal dengan text dibawahnya)



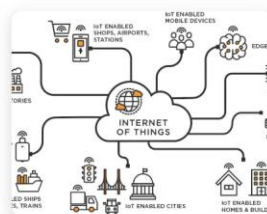
Collections



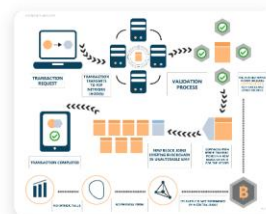
Artificial Intelligence



Software Development



Internet of Things

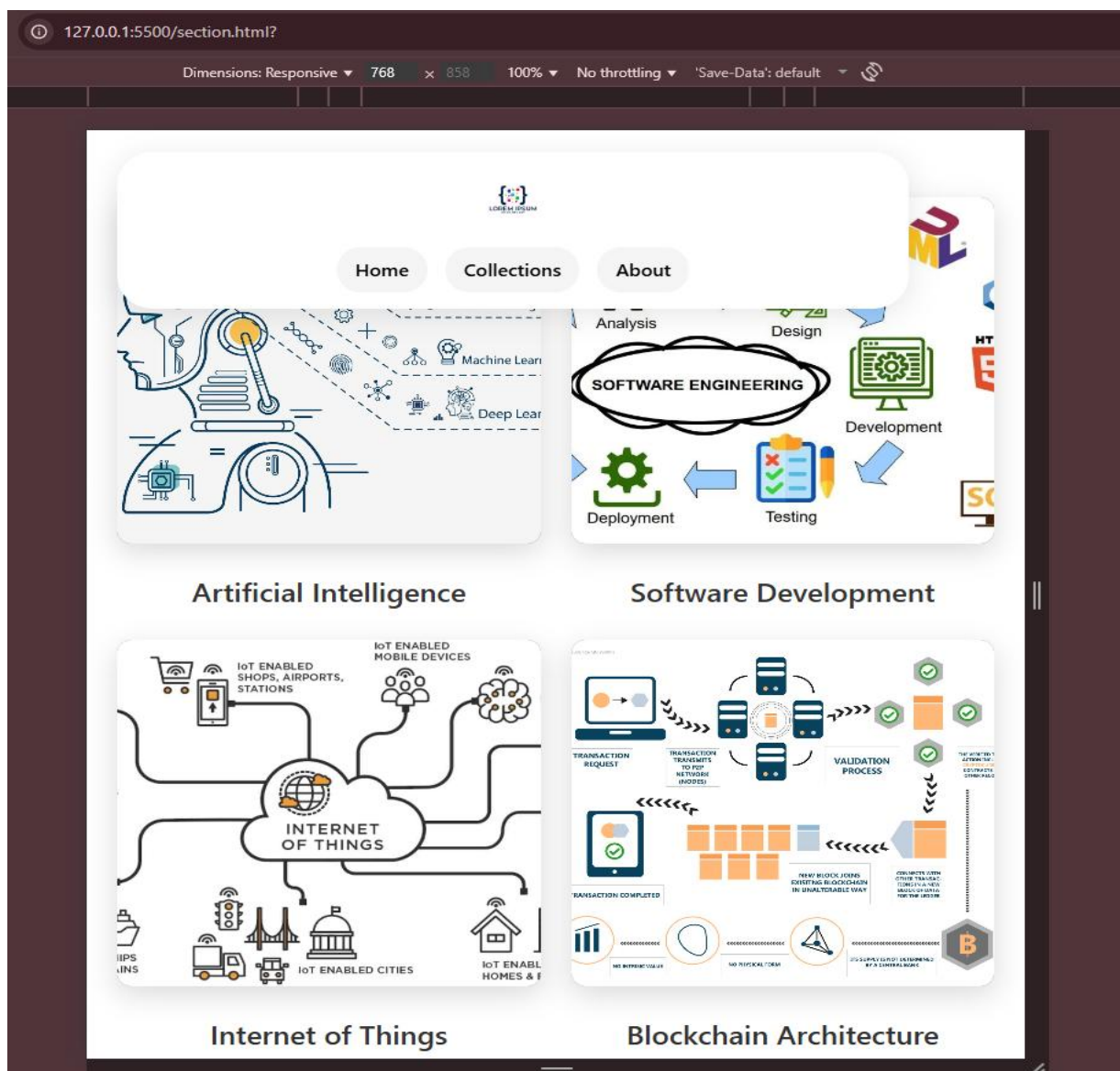


Blockchain Architecture

Jelaskan bagaimana cara anda dapat membuat 4 gambar atau card tersebut memiliki ukuran yang sama

- Untuk membuat empat *card* sama rata, saya menggunakan CSS Grid pada kontainer `.collections-grid` dengan `grid-template-columns: repeat(4, 1fr);` yang membagi ruang menjadi empat kolom berlebar sama. Agar semua gambar sesuai, tinggi kontainer `.card-image` diatur secara tetap (`height: 320px;`) dan tag `<img>` di dalamnya diberi properti `object-fit: cover;` agar gambar mengisi seluruh area *card*. Terakhir, properti `gap: 50px;` digunakan untuk memberi jarak antar *card*.

Kemudian bagaimana jika ada orang yang membuka website yang anda buat di hp dengan ukuran lebar layar kurang dari 768 pixel? Buatlah agar tampilan dapat membuat gambar-gambar dengan maksimum 2 gambar secara horizontal pada layar kurang dari 768 pixel!



Jelaskan apa yang anda gunakan agar styling yang anda berikan dapat berjalan pada ukuran screen tertentu

- Untuk layar di bawah 768px, saya menggunakan *media query* untuk menumpuk navigasi (logo dan menu) secara vertikal. *Layout* koleksi diubah dari beberapa kolom menjadi hanya dua kolom (`grid-template-columns: repeat(2, 1fr)`). Selain itu, ukuran *font* dan *padding* pada berbagai elemen diperkecil agar sesuai dengan layar yang lebih sempit.

```
249
250 /* Responsive Design */
251 @media (max-width: 1200px) {
252   .collections-grid { grid-template-columns: repeat(3, 1fr); gap: 38px; }
253   .home-title { font-size: 58px; }
254   .home-description { font-size: 22px; }
255 }
256
257 @media (max-width: 768px) {
258   .collections-grid { grid-template-columns: repeat(2, 1fr); gap: 25px; }
259   .navbar { padding: 20px 0; }
260   .nav-wrapper { flex-direction: column; gap: 15px; margin: 0 25px; padding: 15px 25px; border-radius: 30px; }
261   .logo { width: 100%; justify-content: center; margin-bottom: 0; }
262   .nav-menu { width: 100%; justify-content: center; gap: 15px; flex-wrap: wrap; }
263   .nav-menu li a { font-size: 16px; padding: 8px 15px; background-color: #f5f5f5; border-radius: 20px; }
264   .home-section { margin: 240px 25px 0; border-radius: 20px; }
265   .collections-section, .content-section { padding: 125px 25px; }
266   .home-content { padding: 50px 25px; }
267   .home-title { font-size: 38px; }
268   .home-description { font-size: 19px; }
269   .section-title { font-size: 42px; }
270   .logo img { height: 58px; }
271   .about-section p { font-size: 20px; }
272   section { scroll-margin-top: 240px; }
273 }
274
275
```